

МИНОБРНАУКИ РОССИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
“ВОРОНЕЖСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ”
(ФГБОУ ВО «ВГУ»)

Факультет компьютерных *наук*

Кафедра программной инженерии в информационных системах

Разработка мобильного приложения для работников автосервиса на платформе
Android

Курсовая работа по дисциплине
Разработка мобильных приложений
6 семестр 2025/2026 учебного года
09.03.02 Информационные системы и технологии

Зав. кафедрой _____ д. ф.-м. н., доцент С.Д. Махортов

Обучающийся _____ ст. 3 курса Р.В. Селютин

Руководитель _____ В.А. Ушаков

Воронеж 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
Определения, обозначения и сокращения.....	5
1 Постановка задачи.....	6
1.1 Обзор аналогов	7
2 Анализ предметной области	11
2.1 Теоретические аспекты предметной области.....	11
3 Реализация	15
3.1 Средства реализации	15
3.2 Логика приложения	17
3.3 Реализация интерфейса.....	23
ЗАКЛЮЧЕНИЕ	30
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	32

ВВЕДЕНИЕ

Проблема. В настоящее время большинство автосервисов ведут учёт задач и ремонтных заявок вручную – с помощью бумажных журналов или таблиц Microsoft Excel. Такой подход приводит к потере данных, дублированию записей, сложностям с отслеживанием статуса выполнения работ и снижению общей эффективности производственного процесса.

Противоречие. Существующие универсальные CRM и ERP-системы автоматизации предоставляют избыточный для рядового механика функционал, требуют дорогостоящего внедрения и обучения, а также зачастую не имеют удобного мобильного интерфейса для использования непосредственно в цехе.

Из вышесказанного следует актуальность разработки специализированного мобильного приложения, целью которого является предоставление работнику автосервиса удобного инструмента для просмотра текущих задач и управления своим рабочим процессом.

Задачи, решаемые для достижения цели:

- Изучить существующий процесс постановки и отслеживания задач в автосервисе;
- Проанализировать существующие аналоги программных продуктов для автоматизации работы автосервиса;
- Определить перечень функциональных требований к разрабатываемому приложению;
- Разработать архитектуру приложения на основе паттерна MVVM;
- Реализовать функции регистрации, аутентификации и просмотра задач в мобильном приложении на платформе Android.

Объект исследования – деятельность работника автосервиса.

Предметом исследования является процесс учёта и отслеживания текущих рабочих задач механиком автосервиса.

Основные разделы: введение, постановка задачи, анализ предметной

области, реализация, заключение.

Область применения –результаты данной работы будут полезны сотрудникам автосервисов, руководителям мастерских, а также разработчикам мобильных приложений для сферы обслуживания автомобилей.

Определения, обозначения и сокращения

В настоящей работе применяются следующие термины с соответствующими определениями:

Мобильное приложение – программное обеспечение, предназначенное для работы на мобильных устройствах (смартфонах и планшетах) под управлением мобильных операционных систем.

Задача (в контексте автосервиса) – заявка на ремонт или техническое обслуживание автомобиля, назначенная конкретному работнику с указанием описания, приоритета и срока выполнения.

REST API – архитектурный стиль взаимодействия компонентов распределённого приложения в сети, основанный на использовании HTTP-методов.

MVVM – архитектурный паттерн «Модель – Представление – Модель представления» (Model – View – ViewModel), обеспечивающий разделение бизнес-логики и пользовательского интерфейса.

БД –база данных.

API –Application Programming Interface, программный интерфейс приложения.

UI –User Interface, пользовательский интерфейс.

ViewModel –компонент архитектуры Android Jetpack, хранящий и обрабатывающий данные для отображения в пользовательском интерфейсе.

Корутина –лёгкий поток выполнения в Kotlin, позволяющий писать асинхронный код в последовательном стиле.

1 Постановка задачи

Объектом разработки является мобильное приложение для работников автосервиса, функционирующее на платформе Android. Приложение предназначено для механиков и других сотрудников производственного подразделения автосервисного предприятия и призвано обеспечить им оперативный доступ к информации о текущих рабочих задачах.

Современный автосервис представляет собой предприятие, в котором одновременно ведётся работа над несколькими десятками автомобилей. Каждый автомобиль проходит несколько этапов обработки: первичная диагностика, согласование перечня работ с клиентом, непосредственное выполнение ремонтных операций, контроль качества и выдача автомобиля. На каждом этапе задействованы разные специалисты: мастер-приёмщик, диагност, механики различных специализаций, маляры, контролёр качества.

В рамках данной работы разрабатывается мобильное приложение, которое позволяет механику видеть список назначенных ему задач непосредственно на экране смартфона. Целью выпускной квалификационной работы является разработка мобильного приложения для платформы Android, позволяющего работнику автосервиса просматривать свои текущие рабочие задачи после авторизации в системе.

К разрабатываемому приложению предъявляются следующие функциональные требования:

- возможность регистрации нового пользователя в системе с указанием электронной почты, пароля и полного имени;
- возможность входа в систему (аутентификации) по электронной почте и паролю;
- отображение списка текущих задач, назначенных авторизованному пользователю;
- отображение детальной информации о каждой задаче: наименование, описание, статус выполнения;
- возможность выхода из учётной записи;

- хранение токена аутентификации для автоматического входа при повторном запуске приложения;
- отображение основной информации о пользователе в разделе настроек.

Нефункциональные требования к приложению:

- приложение должно работать на устройствах под управлением Android версии 7.0 (API level 24) и выше;
- интерфейс приложения должен соответствовать принципам Material Design;
- архитектура приложения должна соответствовать паттерну MVVM для обеспечения поддерживаемости и тестируемости кода;
- взаимодействие с удалённой базой данных должно осуществляться асинхронно, без блокировки главного потока.

1.1 Обзор аналогов

Прежде чем приступить к разработке, был проведён анализ существующих программных продуктов, решающих схожие задачи. Рассмотрено два наиболее распространённых аналога: система «1С: Автодилер» и специализированное решение VINDESK.

«1С: Автодилер» –комплексная информационная система для автоматизации деятельности автодилеров и автосервисных предприятий, разработанная на платформе «1С: Предприятие 8». Система охватывает все аспекты деятельности автосервиса: от первичного приёма автомобиля и формирования заказ-нарядов до учёта запасных частей на складе и формирования управленческой отчётности.

Функциональные возможности «1С: Автодилер» в части управления задачами включают:

- автоматическое формирование заказ-нарядов на основе заявок клиентов;
- назначение работ конкретным механикам с учётом их специализации и текущей загрузки;
- отслеживание статусов выполнения работ в режиме реального времени;

- интеграцию с нормативными справочниками трудозатрат на различные виды ремонта;
- формирование плана загрузки сервисных постов;
- учёт рабочего времени персонала.

Система имеет мобильное приложение для мастера-приёмщика, однако оно ориентировано преимущественно на административный персонал и предоставляет такие функции, как приём заявок, оформление заказ-нарядов и связь с клиентами. Интерфейс для рядового механика в системе практически отсутствует: предполагается, что механик работает за терминалом непосредственно на рабочем посту.

Главным недостатком данного аналога с точки зрения решаемой в данной работе задачи является чрезмерная сложность и высокая стоимость. Внедрение «1С: Автодилер» требует значительных финансовых вложений в лицензии, серверное оборудование и сопровождение. Кроме того, система не предоставляет удобного мобильного интерфейса для механика, работающего в цехе. Наконец, для небольших автосервисов с 5–20 сотрудниками возможности «1С: Автодилер» являются явно избыточными.

Пример интерфейса заказ-наряда в системе «1С: Автодилер» представлен на рисунке 1.

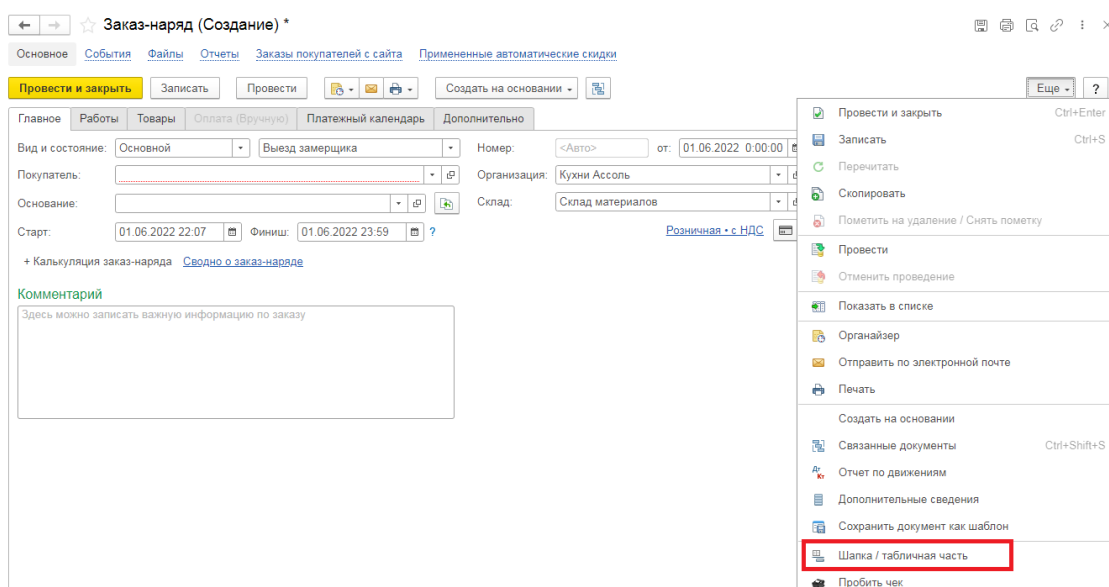


Рисунок 1 - Интерфейс системы «1С: Автодилер». Список заказ-нарядов

VINDESK –специализированная облачная SaaS-платформа (Software as a Service) для управления автосервисом, доступная по подписке. Система спроектирована с учётом потребностей небольших и средних автосервисных предприятий и обладает более современным веб-интерфейсом по сравнению с «1С: Автодилер».

VINDESK предоставляет следующие возможности:

- электронный журнал записи клиентов с интеграцией онлайн-записи;
- управление заказ-нарядами: создание, редактирование, изменение статусов;
- справочник нормативов трудозатрат;
- складской учёт запасных частей и расходных материалов;
- управление персоналом и расчёт зарплаты;
- клиентская база данных с историей обращений;
- SMS и WhatsApp уведомления для клиентов.

Платформа имеет мобильное приложение для iOS и Android, однако оно дублирует функциональность веб-версии и ориентировано на менеджеров, а не на механиков. Для механика в мобильном приложении нет специализированного экрана со списком назначенных задач; вся работа с заявками предполагает полноценный доступ к системе через браузер или планшет.

Существенным недостатком VINDESK является стоимость подписки, которая делает платформу недоступной для совсем небольших автосервисов. Кроме того, система требует постоянного подключения к интернету и зависит от доступности серверов провайдера.

Пример интерфейса VINDESK представлен на рисунке 2.

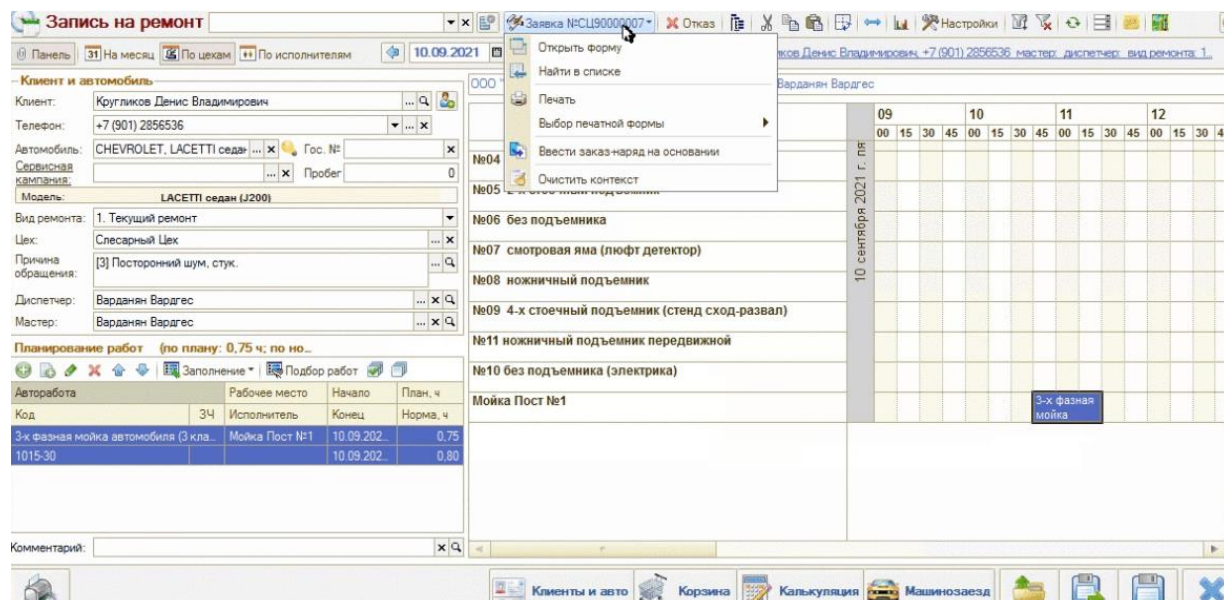


Рисунок 2 - Интерфейс системы VINDESK. Карточка заказ-наряда

Таким образом, главным недостатком рассмотренных аналогов является отсутствие удобного специализированного мобильного интерфейса для механика –рядового работника сервисного цеха. Оба аналога избыточны по функциональности, дорогостоящи и не решают узкую задачу, поставленную в данной работе: предоставить механику на смартфоне список его текущих задач.

2 Анализ предметной области

2.1 Теоретические аспекты предметной области

Автосервис (сервисный центр технического обслуживания и ремонта автомобилей) представляет собой предприятие, оказывающее платные услуги по техническому обслуживанию, диагностике и ремонту транспортных средств. Производственный процесс автосервиса характеризуется высокой степенью параллелизма: одновременно ведутся работы над несколькими автомобилями, каждый из которых находится на определённой стадии выполнения заказа.

Центральным понятием предметной области является задача (рабочая заявка, заказ-наряд) – документ, содержащий информацию о конкретном ремонте или техническом обслуживании автомобиля. Задача включает:

- наименование (краткое описание вида работ);
- детальное описание неисправности или перечня работ;
- сведения об автомобиле (марка, модель, государственный номер, VIN);
- сведения о клиенте;
- назначенного исполнителя (механика);
- текущий статус выполнения (новая, в работе, завершена, приостановлена);
- плановую дату завершения;
- приоритет выполнения.

Жизненный цикл задачи в автосервисе выглядит следующим образом. Мастер-приёмщик принимает автомобиль от клиента, проводит первичную диагностику и формирует заказ-наряд. Далее заявка направляется на авторизацию клиентом (согласование перечня и стоимости работ). После согласования задача назначается конкретному механику и переводится в статус «В работе». По завершении ремонтных работ механик сигнализирует об окончании, задача проходит контроль качества и закрывается.

Для описания процесса формирования и выполнения задачи используются следующие виды нотаций и диаграмм:

- диаграммы вариантов использования (Use Case) –для описания сценариев взаимодействия пользователей с системой;
- диаграммы деятельности –для описания последовательности действий при выполнении рабочего процесса;
- диаграммы классов –для описания структуры данных и связей между объектами.

С точки зрения информационных технологий, задача в автосервисе соответствует концепции «тикета» в системах управления задачами. Подобный подход широко используется в IT-индустрии (системы Jira, YouTrack, Trello) и вполне применим для автоматизации производственных процессов сервисного центра.

Мобильные приложения для Android разрабатываются с использованием Android SDK –набора инструментов и API, предоставляемых компанией Google. Разработка ведётся преимущественно на языках Kotlin или Java. Язык Kotlin является официально рекомендованным языком разработки для Android-приложений начиная с 2019 года [1].

Архитектурные паттерны имеют важное место в разработке Android-приложений. Наиболее распространённые из них [2]:

- MVC (Model-View-Controller) –исторически первый паттерн, в котором контроллер управляет взаимодействием модели и представления; в контексте Android контроллером выступает Activity;
- MVP (Model-View-Presenter) –паттерн, в котором Presenter является посредником между моделью и представлением, что упрощает тестирование;
- MVVM (Model-View-ViewModel) –рекомендованный Google паттерн, в котором ViewModel предоставляет данные для представления с помощью реактивных потоков данных.

В разрабатываемом приложении используется паттерн MVVM, который является официально рекомендованным для Android-разработки. Данный паттерн обеспечивает чёткое разделение ответственности между

компонентами приложения и упрощает как тестирование, так и последующее сопровождение кода.

В качестве серверной части (бэкенда) для хранения данных в разрабатываемом приложении выбрана платформа Supabase. Supabase –это платформа типа BaaS (Backend as a Service), предоставляющая разработчикам готовую инфраструктуру на базе реляционной СУБД PostgreSQL. Supabase предоставляет REST API для взаимодействия с базой данных, систему аутентификации пользователей, хранилище файлов и функции реального времени [3].

PostgreSQL –мощная объектно-реляционная СУБД с открытым исходным кодом, обладающая высокой производительностью и широкими возможностями. В контексте данной работы PostgreSQL используется для хранения информации о пользователях и задачах.

Структура базы данных разрабатываемого приложения включает следующие основные таблицы:

- таблица `users` –расширенная информация о пользователях системы (полное имя, должность, дата регистрации, ссылка на аутентификационную запись Supabase Auth);
- таблица `tasks` –задачи, назначенные сотрудникам (идентификатор, наименование, описание, статус, идентификатор назначенного исполнителя, дата создания, плановая дата завершения).

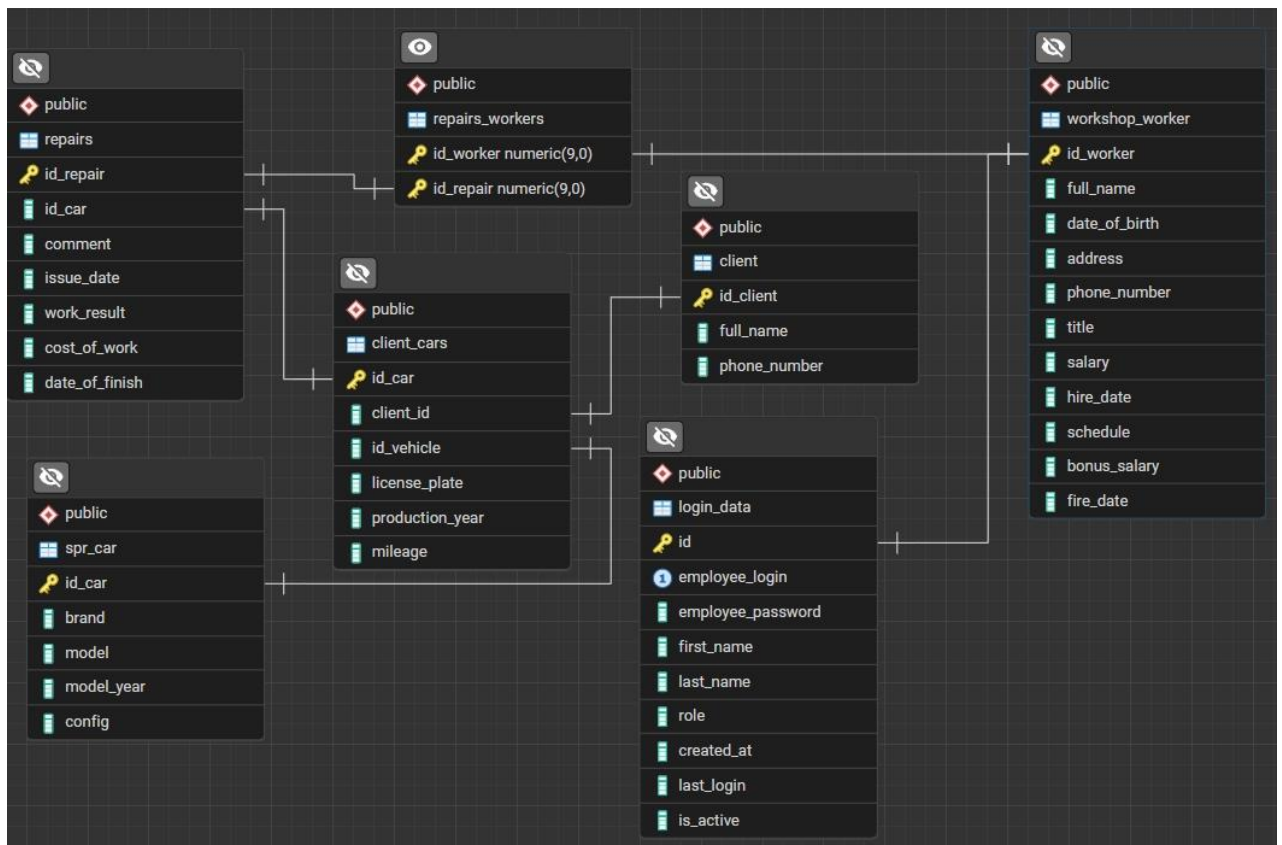


Рисунок 3 - Схема базы данных приложения

Для взаимодействия мобильного приложения с REST API Supabase используется библиотека Retrofit –типобезопасный HTTP-клиент для Android, разработанный компанией Square. Retrofit позволяет описывать API-эндпоинты в виде интерфейсов Kotlin, что обеспечивает строгую типизацию запросов и ответов [4].

Асинхронное выполнение сетевых запросов реализуется с помощью Kotlin Coroutines –механизма лёгких потоков, встроенного в язык Kotlin. Корутины позволяют писать асинхронный код в последовательном стиле, что значительно упрощает понимание логики приложения по сравнению с классическими callback-подходами [5].

3 Реализация

3.1 Средства реализации

Для разработки мобильного приложения использован следующий набор технологий и инструментов:

- язык программирования Kotlin;
- среда разработки Android Studio;
- архитектурный паттерн MVVM (Model-View-ViewModel);
- библиотека Retrofit для выполнения HTTP-запросов;
- платформа Supabase для серверной части (СУБД PostgreSQL);
- Kotlin Coroutines для асинхронного программирования;
- компонент ViewModel из Android Jetpack;
- SharedPreferences для локального хранения токена аутентификации;
- библиотека Gson для сериализации/десериализации JSON.

Язык программирования Kotlin –статически типизированный современный язык, разработанный компанией JetBrains и официально поддерживаемый Google в качестве основного языка разработки Android-приложений. Kotlin обладает следующими преимуществами по сравнению с Java [1]:

- краткость синтаксиса: типичный Kotlin-код на 40% короче эквивалентного Java-кода;
- нулевая безопасность (Null Safety): система типов языка разделяет ссылки, которые могут содержать null, и те, которые не могут; это позволяет обнаруживать ошибки, связанные с нулевыми ссылками, на этапе компиляции;
- полная совместимость с Java: Kotlin-код может использовать все существующие Java-библиотеки;
- встроенная поддержка корутин для асинхронного программирования;
- функции-расширения: возможность добавлять новые методы к существующим классам без наследования;
- лямбда-выражения и функции высшего порядка.

Android Studio –официальная интегрированная среда разработки для Android-приложений, разработанная компанией Google на основе IntelliJ IDEA. Среда разработки предоставляет: визуальный редактор макетов пользовательского интерфейса, встроенный эмулятор Android-устройств, инструменты профилировщика производительности, систему управления зависимостями Gradle, поддержку системы контроля версий Git [2].

Supabase –открытая платформа типа Backend as a Service, позиционируемая как альтернатива Firebase от Google. В основе платформы лежит реляционная СУБД PostgreSQL, что обеспечивает надёжное хранение данных и мощные возможности для выборки [3]. Supabase предоставляет:

- автоматически генерируемый RESTful API для всех таблиц базы данных;
- встроенную систему аутентификации пользователей с поддержкой email/password, OAuth-провайдеров;
- возможности для реализации подписок на изменения данных в реальном времени;
- JavaScript SDK и SDK для различных языков программирования, а также чистый REST API для любых платформ;
- веб-интерфейс для управления базой данных и мониторинга.

Retrofit –типобезопасная библиотека HTTP-клиента для Android, разработанная компанией Square. Библиотека позволяет описывать HTTP-запросы в виде Kotlin/Java-интерфейсов с аннотациями, что делает код взаимодействия с API лаконичным и понятным. Retrofit автоматически выполняет сериализацию и десериализацию данных, поддерживает работу с корутинами Kotlin. Среди преимуществ Retrofit: простота настройки, широкое сообщество разработчиков, хорошая документация, поддержка различных конвертеров (Gson, Moshi, Jackson) [4].

Kotlin Coroutines –встроенный в язык Kotlin механизм для асинхронного программирования, реализующий концепцию «последовательного асинхронного кода». Корутины позволяют выполнять длительные операции

(сетевые запросы, работа с базой данных) в фоновом потоке, не блокируя главный (UI) поток приложения. При этом код выглядит так же, как обычный последовательный код, что значительно упрощает его чтение и отладку [5].

3.2 Логика приложения

Приложение построено в соответствии с архитектурным паттерном MVVM (Model-View-ViewModel), который является официально рекомендованным Google подходом к организации кода Android-приложений. Паттерн MVVM обеспечивает чёткое разделение ответственности между тремя слоями [2]:

- Model (Модель) —отвечает за бизнес-логику и данные. Включает классы данных, репозитории и Use Case'ы. Взаимодействует с удалённой базой данных через Retrofit и REST API Supabase;
- View (Представление) —отвечает за отображение данных и взаимодействие с пользователем. Представлено активностями (Activity) и фрагментами (Fragment). Наблюдает за изменениями данных через механизмы LiveData и StateFlow, не содержит бизнес-логики;
- ViewModel (Модель представления) —служит посредником между Model и View. Хранит состояние экрана, переживает повороты устройства, обрабатывает действия пользователя и предоставляет данные View через реактивные потоки.

Дополнительно в архитектуре применяется принцип Clean Architecture: между ViewModel и репозиториями вводится слой Use Case'ов. Каждый Use Case отвечает за одно конкретное действие бизнес-логики (например, GetTasksUseCase —получение списка задач, SignInUseCase —вход в систему). Такой подход делает каждый компонент независимым и легко тестируемым.

Взаимодействие компонентов при загрузке списка задач происходит следующим образом:

- пользователь открывает экран со списком задач (Fragment);
- Fragment обращается к ViewModel и инициирует загрузку данных;
- ViewModel вызывает GetTasksUseCase;

- `GetTasksUseCase` обращается к `TaskRepository`;
- `TaskRepository` выполняет HTTP-запрос к REST API Supabase через `Retrofit` в контексте `IO-диспетчера` корутины;
- полученные данные возвращаются обратно по цепочке: `Repository` → `UseCase` → `ViewModel`;
- `ViewModel` обновляет `UiState` (состояние экрана), что автоматически приводит к обновлению UI во `Fragment`.

В результате разработки приложение состоит из следующих пакетов и модулей:

Пакет `data` содержит классы, отвечающие за работу с данными:

- `SupabaseApi.kt` –`Retrofit`-интерфейс, описывающий все конечные точки REST API;
- `RetrofitClient.kt` –синглтон для инициализации и хранения экземпляра `Retrofit`;
- `model/` –классы данных (`AuthRequest`, `AuthResponse`, `User`, `Task`, `UserInfo`);
- `repository/` –реализации репозиториев (`AuthRepositoryImpl`, `TaskRepositoryImpl`).

Пакет `domain` содержит интерфейсы репозиториев и Use Case'ы:

- `repository/` –интерфейсы `AuthRepository` и `TaskRepository`;
- `usecase/` –классы `GetTasksUseCase`, `SignInUseCase`, `SignUpUseCase`, `LogoutUseCase`, `GetCurrentUserUseCase`.

Пакет `ui` содержит классы пользовательского интерфейса, организованные по экранам:

- `auth/` –`LoginActivity`, `RegisterActivity` и соответствующие `ViewModel`'и;
- `home/` –`MainActivity` с навигационным `BottomNavigationView`;
- `tasks/` –`TasksFragment` и `TasksViewModel`;
- `settings/` –`SettingsFragment` и `SettingsViewModel`.

Интерфейс `SupabaseApi.kt` описывает все HTTP-эндпоинты взаимодействия с Supabase. В листинге 1 представлена его реализация.

Листинг 1 – Интерфейс SupabaseApi.kt

```
interface SupabaseApi {

    companion object {

        const val PUBLISHABLE_KEY = "sb_publishable_..."

    }

    @POST("auth/v1/signup")

    @Headers("apikey: $PUBLISHABLE_KEY")

    suspend fun signUp(

        @Body request: AuthRequest

    ): AuthResponse

    @POST("auth/v1/token")

    @Headers("apikey: $PUBLISHABLE_KEY")

    suspend fun signIn(

        @Query("grant_type") grantType: String = "password",

        @Body request: AuthRequest

    ): AuthResponse

    @POST("auth/v1/logout")

    @Headers("apikey: $PUBLISHABLE_KEY")

    suspend fun logout(

        @Header("Authorization") token: String

    ): Unit

    @GET("auth/v1/user")

    @Headers("apikey: $PUBLISHABLE_KEY")

    suspend fun getUser(

        @Header("Authorization") token: String

    ): UserResponse

}
```

```

@GET("rest/v1/tasks")

@Headers("apikey: $PUBLISHABLE_KEY")

suspend fun getTasks(

    @Header("Authorization") token: String,

    @Query("assigned_to") eq: String? = null

): List<Task>

}

```

Репозиторий аутентификации `AuthRepositoryImpl` реализует паттерн `Repository` и инкапсулирует логику работы с API аутентификации. В листинге 2 представлена реализация метода `signIn`.

Листинг 2 – Реализация метода `signIn` в `AuthRepositoryImpl`

```

override suspend fun signIn(

    email: String,

    password: String

): Result<AuthResponse> {

    return withContext(Dispatchers.IO) {

        try {

            val request = AuthRequest(email, password)

            val response = api.signIn(request = request)

            saveToken(response.accessToken)

            Result.success(response)

        } catch (e: Exception) {

            Result.failure(e)

        }

    }

}

```

Для хранения токена аутентификации между сессиями используется механизм `SharedPreferences` – стандартный инструмент Android для хранения небольших объёмов данных. Токен сохраняется при успешном входе и очищается при выходе из аккаунта. В листинге 3 показана реализация методов работы с токеном.

Листинг 3 – Реализация методов работы с токеном

```
override fun saveToken(token: String) {  
    sharedPreferences.edit()  
        .putString("auth_token", token)  
        .apply()  
}  
  
override fun getToken(): String? {  
    return sharedPreferences.getString("auth_token", null)  
}  
  
override fun clearSession() {  
    sharedPreferences.edit()  
        .remove("auth_token")  
        .apply()  
}
```

`ViewModel` каждого экрана хранит состояние в виде `UiState` – data-класса, содержащего все данные, необходимые для отображения экрана. Состояние обновляется через `MutableStateFlow`, а `Fragment` подписывается на его изменения через `observe`. В листинге 4 приведена реализация `TasksViewModel` [7].

Листинг 4 – Реализация `TasksViewModel`

```
class TasksViewModel(  
    private val getTasksUseCase: GetTasksUseCase
```

```

) : ViewModel() {

    private val _uiState = MutableStateFlow(TasksUiState())

    val uiState: StateFlow<TasksUiState> =
        _uiState.asStateFlow()

    fun loadTasks() {

        viewModelScope.launch {

            _uiState.update { it.copy(isLoading = true) }

            getTasksUseCase().fold(

                onSuccess = { tasks ->

                    _uiState.update {

                        it.copy(tasks = tasks, isLoading =
false)

                    }

                },

                onFailure = { error ->

                    _uiState.update {

                        it.copy(error = error.message, isLoading
= false)

                    }

                }

            )

        }

    }

}

```

SettingsViewModel отвечает за отображение информации о текущем пользователе и функцию выхода из аккаунта. При инициализации ViewModel загружает данные пользователя из репозитория. В листинге 5 показана

реализация функции выхода.

Листинг 5 – Реализация функции выхода из аккаунта

```
fun logout() {  
  
    viewModelScope.launch {  
  
        _uiState.update { it.copy(isLoading = true) }  
  
        val token = authRepository.getToken() ?: ""  
  
        authRepository.logout(token)  
  
        authRepository.clearSession()  
  
        _uiState.update {  
  
            it.copy(isLoading = false, isLoggedOut = true)  
  
        }  
  
    }  
  
}
```

3.3 Реализация интерфейса

Пользовательский интерфейс приложения реализован с использованием XML-разметки и стандартных компонентов Material Design [7]. Навигация между экранами организована с помощью компонента Navigation Component из библиотеки Android Jetpack [6].

Приложение включает следующие экраны:

- экран входа (LoginActivity) –поля ввода email и пароля, кнопка «Войти», ссылка на экран регистрации;
- экран регистрации (RegisterActivity) –поля для ввода полного имени, email и пароля, кнопка «Зарегистрироваться»;
- главный экран (MainActivity) с нижней панелью навигации –содержит два фрагмента: TasksFragment и SettingsFragment;
- фрагмент задач (TasksFragment) –список текущих задач пользователя, отображаемый с помощью RecyclerView;

— фрагмент настроек (SettingsFragment) – информация о пользователе и кнопка выхода.

При первом запуске приложения проверяется наличие сохранённого токена аутентификации. Если токен найден, пользователь сразу перенаправляется на главный экран. В противном случае отображается экран входа. Данная логика реализована в SplashActivity, которая является точкой входа в приложение.

Процесс аутентификации начинается с экрана входа. Пользователь вводит адрес электронной почты и пароль, после чего нажимает кнопку «Войти». В этот момент отображается индикатор загрузки, а сетевой запрос к API Supabase выполняется в фоновом потоке. Экран входа представлен на рисунке 4.

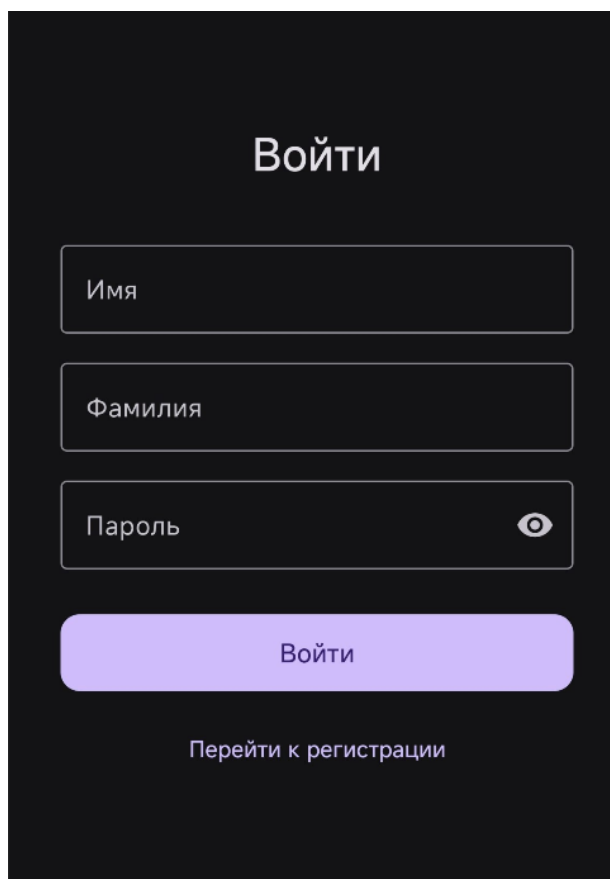
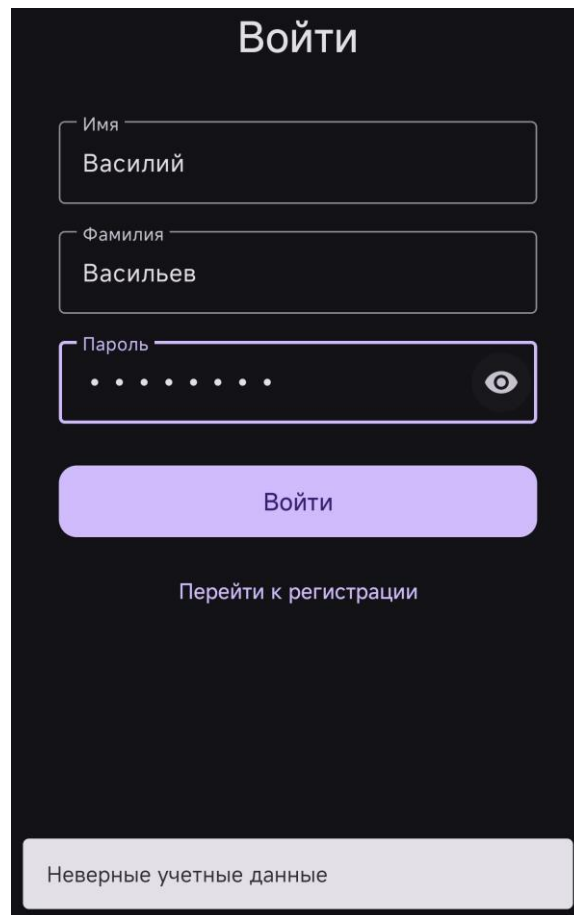


Рисунок 4 - Экран входа в приложение

При вводе некорректных данных или при ошибке сети под полями ввода отображается текстовое сообщение об ошибке, как показано на рисунке 5.



Войти

Имя
Василий

Фамилия
Васильев

Пароль
.....

Войти

[Перейти к регистрации](#)

Неверные учетные данные

Рисунок 5 - Экран входа. Сообщение об ошибке аутентификации

Экран регистрации позволяет новому пользователю создать учётную запись. Для регистрации необходимо указать полное имя (ФИО), адрес электронной почты и пароль. Реализована валидация вводимых данных: проверяется корректность формата email, длина пароля (не менее 6 символов) и непустота поля имени. Экран регистрации представлен на рисунке 6.

Регистрация

Имя

Фамилия

Электронная почта

Номер телефона

Пароль

Регистрация

Перейти к входу

Рисунок 6 - Экран регистрации нового пользователя

После успешной регистрации пользователь автоматически входит в систему и перенаправляется на главный экран, минуя экран входа.

Главный экран приложения использует компонент `BottomNavigationView` с двумя разделами: «Задачи» и «Настройки». По умолчанию открывается раздел «Задачи». Переключение между разделами не вызывает повторной загрузки данных, так как оба фрагмента сохраняют своё состояние через `ViewModel`.

Раздел «Задачи» (`TasksFragment`) отображает список задач, назначенных текущему пользователю. Список реализован с помощью `RecyclerView` и специализированного адаптера `TaskAdapter`. Каждый элемент списка представлен карточкой (`CardView`), содержащей:

- наименование задачи (жирный шрифт);
- краткое описание (до 2 строк);

— индикатор статуса (цветная метка): зелёный —завершена, синий —в работе, серый —новая.

Экран со списком задач представлен на рисунке 7.

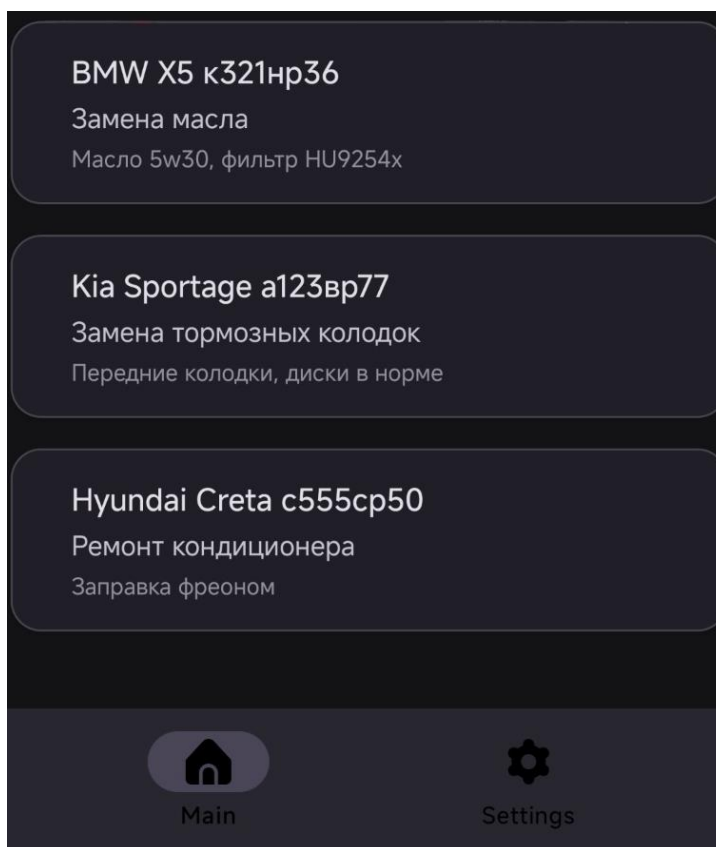


Рисунок 7 - Экран «Задачи». Список текущих задач механика

Раздел «Настройки» (SettingsFragment) отображает основную информацию об авторизованном пользователе: полное имя, адрес электронной почты и дату регистрации в системе. Данные загружаются из API при каждом открытии раздела. Также в разделе «Настройки» располагается кнопка «Выйти из аккаунта». При нажатии на кнопку отображается диалоговое окно подтверждения, что предотвращает случайный выход. Экран настроек представлен на рисунке 8.

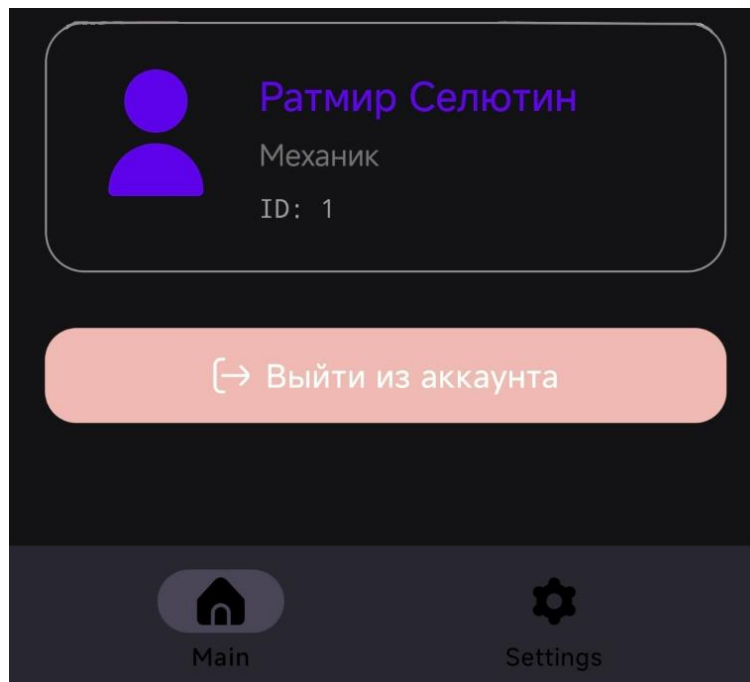


Рисунок 8 - Экран «Настройки». Информация о пользователе

После подтверждения выхода токен аутентификации удаляется из SharedPreferences, выполняется запрос к API Supabase для инвалидации токена на стороне сервера, и пользователь перенаправляется на экран входа. Диалоговое окно подтверждения выхода представлено на рисунке 9.

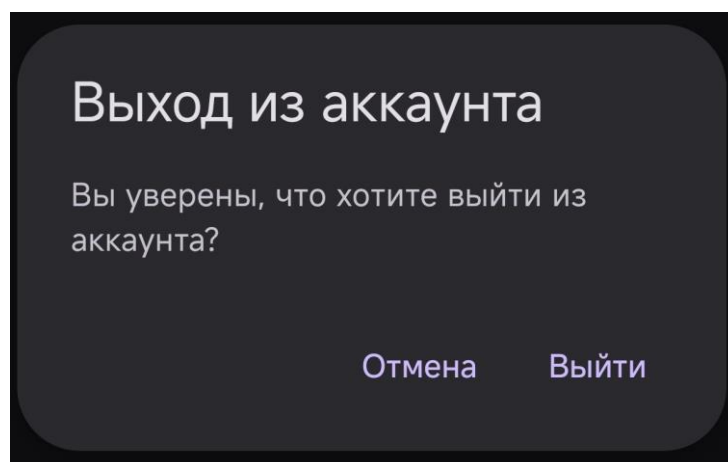


Рисунок 9 - Диалоговое окно подтверждения выхода из аккаунта

Диаграмма вариантов использования (Use Case Diagram) описывает взаимодействие пользователей (актеров) с системой. В разрабатываемом приложении выделяется два актора:

- Незарегистрированный пользователь –пользователь, который ещё не имеет учётной записи в системе;
- Авторизованный пользователь (Работник) –сотрудник автосервиса, успешно прошедший аутентификацию.

Незарегистрированный пользователь может выполнять следующие действия:

- регистрация в системе –создание новой учётной записи с указанием имени, email и пароля;
- вход в систему –аутентификация по email и паролю.

Авторизованный работник получает доступ к следующим функциям:

- просмотр списка задач –отображение всех задач, назначенных текущему пользователю;
- просмотр информации о своём аккаунте –имя, email, дата регистрации;
- выход из аккаунта –завершение сессии и переход к экрану входа.

Таким образом, разработанный механик по продажам благодаря созданному мобильному приложению получил следующие возможности:

зарегистрироваться в системе и авторизоваться при каждом последующем запуске;

в любой момент видеть актуальный список назначенных задач прямо на экране смартфона;

просматривать описание каждой задачи и её статус выполнения;

просматривать информацию о своей учётной записи;

выходить из аккаунта при передаче устройства другому сотруднику.

ЗАКЛЮЧЕНИЕ

В результате выполнения выпускной квалификационной работы было разработано мобильное приложение для платформы Android, предназначенное для работников автосервиса и позволяющее им просматривать список текущих рабочих задач.

В ходе работы был изучен процесс учёта задач в автосервисе. Было установлено, что большинство небольших и средних автосервисных предприятий ведут учёт рабочих заявок вручную –с помощью бумажных журналов или электронных таблиц. Это приводит к потере данных, задержкам в информировании сотрудников о новых задачах и снижению общей эффективности производственного процесса.

Были проанализированы существующие программные аналоги: система «1С: Автодилер» и облачная платформа VINDESK. Оба рассмотренных решения обладают избыточной для рядового механика функциональностью, ориентированы преимущественно на административный персонал и не предоставляют удобного специализированного мобильного интерфейса для работника производственного подразделения.

На основе анализа предметной области и существующих аналогов были сформулированы функциональные требования к разрабатываемому приложению, в соответствии с которыми были реализованы следующие возможности:

- регистрация нового пользователя в системе с указанием электронной почты, пароля и полного имени;

- аутентификация пользователя по электронной почте и паролю с сохранением токена сессии;

- отображение списка текущих задач, назначенных авторизованному пользователю, с указанием наименования, описания и статуса каждой задачи;

- просмотр информации о текущем пользователе в разделе настроек;

- возможность выхода из учётной записи.

При разработке приложения использованы современные технологии: язык Kotlin, архитектурный паттерн MVVM, библиотека Retrofit для работы с REST API, платформа Supabase с СУБД PostgreSQL в качестве серверной части, Kotlin Coroutines для асинхронного выполнения сетевых запросов.

Применение паттерна MVVM обеспечило чёткое разделение ответственности между компонентами приложения, упростило тестирование бизнес-логики и повысило общую поддерживаемость кода. Использование Supabase позволило быстро создать надёжную серверную инфраструктуру без необходимости самостоятельной разработки и развёртывания бэкенда.

Разработанное приложение решает реальную практическую задачу: предоставляет механику автосервиса удобный мобильный инструмент для получения информации о своих рабочих задачах без необходимости обращения к администраторскому терминалу. Приложение может быть масштабировано: возможно добавление функции обновления статуса задачи, push-уведомлений о новых назначениях, истории выполненных работ и других возможностей.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Скин, Д. Android-разработка с Kotlin / Д. Скин, К. Гарднер. – Москва: ДМК Пресс, 2022. –496 с. –ISBN 978-5-93700-141-5.
2. Официальная документация Google Android Developers: Guide to app architecture [Электронный ресурс]. –URL: <https://developer.android.com/topic/architecture> (дата обращения: 15.04.2026).
3. Официальная документация Supabase [Электронный ресурс]. – URL: <https://supabase.com/docs> (дата обращения: 20.04.2026).
4. Документация библиотеки Retrofit [Электронный ресурс]. – URL: <https://square.github.io/retrofit> (дата обращения: 18.04.2026).
5. Официальная документация Kotlin. Kotlin Coroutines [Электронный ресурс]. –URL: <https://kotlinlang.org/docs/coroutines-guide.html> (дата обращения: 15.04.2026).
6. Официальная документация Android Jetpack: ViewModel Overview [Электронный ресурс]. –URL: <https://developer.android.com/topic/libraries/architecture/viewmodel> (дата обращения: 18.04.2026).
7. Официальная документация Material Design [Электронный ресурс]. –URL: <https://m3.material.io> (дата обращения: 22.04.2026).